MEGAZONECLOUD

Kiro 와 함께하는 AI 챗봇 Build-Up

2026 2H GenAI Lab

MEGAZONE CLOUD
김예지, 최재혁 매니저

INDEX

목차

01 RAG 아키텍처 개요 및 Bedrock Knowledge Bases

02 Kiro IDE — AI 주도 인프라 배포 자동화

03 KB 워크플로우 및 데이터 파이프라인

04 심화 : 커스텀 IDP 구축 전략

05 Wrap-up 및 실습 안내

PART 01

RAG 아키텍처 개요 및 Bedrock KB

RAG의 개념 및 Knowledge Base 아키텍처

핵심 LLM의 입력 용량 제한을 극복하기 위해, 사내 데이터를 사전 색인(Indexing)하고 관련 문맥만 동적으로 추출

기존 LLM의 한계 (프롬프트 주입 방식)

"매번 책 한 권을 통째로 읽어줘"

- 컨텍스트 윈도우(입력 용량) 제한으로 대규모 문서 처리 불가
- 일회성 정보 주입으로 세션 종료 시 데이터 휘발

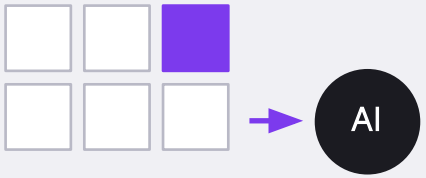


매번 통째로 ×N

지식 기반(Knowledge Base) 아키텍처

"색인해 두고, 맞는 페이지만 펼친다"

- 사내 데이터의 영구적 지식 자산화 (벡터화)
- 환각(Hallucination) 방지 및 정확한 출처(Source) 추적



필요한 조각만 꺼냄

자동화된 데이터 전처리 파이프라인 (Data Pipeline)

1 데이터 추출 Parsing
문서에서 글자·표를 꺼냄

→

2 텍스트 분할 Chunking
검색하기 좋은 조각으로

→

3 수치화 Embedding
의미를 담은 벡터로

→

4 벡터 DB Vector Store
데이터를 보관할 전용 창고를 연결

→

5 정보 검색 *Retrieval*
질문하면 관련 조각만 꺼냄

RAG 시스템 구축을 위한 아키텍처 역할 분담

핵심 복잡한 인프라와 데이터 파이프라인은 AWS 완전 관리형으로 위임하고, 개발자는 비즈니스 로직 구현에 집중

우리가 할 일

서비스 기획 및 프론트엔드 통합

- 데이터 처리 파라미터(청킹, 임베딩 모델 등) 설정
- UI/UX 구현 및 기업용 인증/보안 계층 통합

+

AWS가 할 일

완전 관리형 RAG 백엔드 운영

- 데이터 추출, 청킹, 벡터화 및 인덱싱 전 과정 자동화
- 인프라 관리 부담이 없는 서버리스(Serverless) 환경 제공

+

Kiro가 할 일

배포 자동화

- 자연어 명령어 기반의 클라우드 자동 배포
- 배포 중 오류 발생 시 자가 치유(Self-healing) 및 자동 복구

이 프로젝트가 고른 옵션

데이터 추출	텍스트 분할	수치화	저장소	검색
Claude가 눈으로 읽음 (표·이미지 OK)	의미 단위로 자름 (최대 500토큰)	Titan V2 임베딩 (1024차원)	OpenSearch Serverless	의미+키워드 (HYBRID)

End-to-End 서비스 워크플로우

핵심 문서 업로드부터 질의응답까지, 사용자 개입을 최소화한 완전 자동화(Fully Automated) 파이프라인을 제공

우리

문서 올리기

Data Upload

S3 이벤트 기반의 자동 동기화

완전 관리형 데이터 처리 및 추론 (AWS Managed)

읽기 → 자르기 → 숫자 변환 → 저장·검색

사용자 질문에 맞춘 **관련 컨텍스트 자동 추출** 및 환각이 통제된 답변 생성

우리

실시간 질의응답

Q&A

신뢰도 점수 및 원본 데이터 출처 명시

고급 검색 기능 : 데이터 적재 시 대형 언어 모델(LLM)이 ***메타데이터 (주제, 연도 등)***를 자동 추출 및 태깅하여, 향후 다차원 조건부 필터링 검색을 지원합니다.

PART 02

Kiro IDE — AI 주도 인프라 배포 자동화

Kiro IDE 개요: 문서 기반 자율 에이전트 개발 환경

핵심 단순 코드 생성을 넘어, 3가지 핵심 정의 문서를 기반으로 SDLC 전 과정을 오케스트레이션

AWS 기반 Agentic IDE: 단순 챗봇 형태의 코딩 어시스턴트가 아닌, 요구사항 분석부터 아키텍처 설계, 구현, 테스트 및 검증에 이르는 End-to-End 워크플로우를 자율적으로 수행합니다.

스펙 Specs

Task-specific 명세서

특정 프로젝트의 요구사항, 아키텍처 설계, 태스크 분할(WBS)을 정의한 문서 — Kiro는 이를 바탕으로 자율적인 실행 계획을 수립

`.kiro/specs/` · 예: UI 리디자인 스펙

스티어링 Steering

프로젝트 전역(Global) 규칙

기술 스택, 코딩 표준, 보안 규정 등을 정의 — Kiro의 모든 코드 생성 및 작업 흐름에 강제되는 아키텍처 기준점

`.kiro/steering/` · 프로젝트의 상시 규칙

스킬 Skills

표준 작업 절차(SOP) 정의

인프라 배포, 테스트 환경 구축 등 반복되는 파이프라인의 실행 절차를 정의하여, 특정 이벤트 발생 시 Kiro가 자체적으로 트리거하고 수행

`.kiro/skills/` · 예: `ec2-ssm-cdk-deploy(배포)`

Kiro

Context-Aware 자율 실행 엔진
: 정의된 메타데이터(Steering, Skills, Specs)를 컨텍스트로 결합하여, 자연어 명령 하나로 '계획 수립 → 코드 실행 → 검증'의 자율 루프를 완성

Kiro로 자율형 인프라 프로비저닝 구현

핵심 자연어 명령 기반으로 인프라 구성, 배포, 통합 테스트 및 오류 복구 전 과정을 Kiro 에이전트가 오케스트레이션

STEP 1 · 준비

1 필수 파라미터 수집

AWS 계정, 타겟 리전, 관리자 권한 등 배포에 필요한 최소한의 컨텍스트만 동적으로 요구

2 환경 사전 점검

기존 클라우드 환경의 형상 불일치(Drift)를 파악하고 배포 전 스스로 정상 상태로 복구

3 빌드 인스턴스(EC2) 자동 프로비저닝

보안 터널링(SSM)을 통해 SSH 키 없이 안전하게 접근 가능한 임시 빌드 서버를 구축

STEP 2 · 배포

4 아티팩트(Artifact) 패키징 및 업로드

로컬 환경의 최신 소스 코드를 압축하여 타겟 빌드 환경으로 안전하게 전송

5 리전 특화 리소스 동적 매핑

AWS 리전별로 가용성이 다른 AI 모델 및 리소스 설정을 환경에 맞게 자동 설정

6 IaC 기반 일괄 배포

종속성 설치부터 소스 빌드, `cdk deploy` 실행까지 IaC 배포 자동 수행

STEP 3 · 마무리

7 오류 자동 수정

배포 중 에러 발생 시, Kiro가 에러 로그를 분석하여 코드를 직접 수정하고 재시도(최대 5회)를 수행

8 통합 테스트 수행

인증, 문서 적재, RAG 질의응답 등 핵심 비즈니스 로직이 서버에서 정상 동작하는지 자동 검증

9 리소스 상태 추적 및 롤백

프로비저닝된 모든 리소스의 형상을 기록하여, 추후 완전하고 깔끔한 리소스 Destroy를 지원

PART 03

KB 워크플로우 및 데이터 파이프라인

RAG 시스템의 핵심 워크플로우

핵심 무거운 데이터 적재(Ingestion) 워크플로우와 가벼운 실시간 검색(Retrieval) 파이프라인을 분리하여 시스템 효율을 최적화

한 번만 — 문서를 넣을 때 (색인 준비)

1. **올리면 자동 시작**
   S3에 올리는 순간 색인 작업이 알아서 시작
2. **Claude가 '눈으로' 읽음**
   표·이미지·스캔본도 페이지를 보고 이해 (비전 파싱)
3. **의미 단위로 자름**
   화제가 바뀌는 지점에서 자름 — 문맥이 안 끊김
4. **벡터로 저장**
   숫자로 변환해 OpenSearch에 색인 (+꼬리표 부착)

매번 — 질문할 때 (검색 + 답변)

1. **질문 (+옵션 선택)**
   모델·말투·안전장치를 그때그때 교체 — 재배포 불필요
2. **컨텍스트 검색**
   질문의 의도와 문맥을 분석하여 가장 정확한 근거 데이터를 추출
3. **근거를 주고 답 생성**
   찾은 조각을 Claude에 건네 그 근거로만 답하게 함
4. **출처·기억 붙이기**
   출처 번호·관련도 점수 표시, 대화 맥락은 DB에 저장

실습 아키텍처 — 사용자 질의 처리 파이프라인

핵심 프론트엔드부터 AI 검색 엔진까지, 4개의 독립적인 모듈(CDK Stack)으로 구성된 서버리스 아키텍처



AWS 완전 관리형 영역

- 데이터 적재 및 하이브리드 검색 파이프라인 (추출, 청킹, 임베딩, 인덱싱 자동화)
- 인프라 프로비저닝 없이 API 파라미터 설정만으로 백엔드 통제

커스텀 비즈니스 로직 영역

- 인터페이스(React), 사용자 인증(Cognito), 세션 상태 관리(DynamoDB) 구현
- LLM 제어: 가드레일(보안 정책), 시스템 프롬프트(페르소나 부여), 출처 표시

데이터 분할(Chunking) 전략 및 컨텍스트 보존 아키텍처

핵심 세그멘테이션 불균형에 따른 문맥 손실과 노이즈를 방지하기 위해, 데이터 특성에 맞는 최적의 청킹 전략 설계 필요

자르는 방법 4가지

1 고정 크기

정량적 토큰 기준 분할 — 문맥 끊기 및 의미 훼손 위험 존재

✓ 이 프로젝트의 선택

2 의미 기반



의미론적 분기점 기반 동적 분할 — 문맥의 연속성 보존 최적화

3 계층 기반



부모-자식 컨텍스트 매핑 — 정밀도 향상 및 인덱스 구조 복잡도 증가

4 자르지 않음



단일 문서 전체 인덱싱 — 짧은 문서 데이터 전용

선택한 설정과 알아둘 것

이 프로젝트의 설정

최대 500토큰 ≈ 한 문단

문장 간 의미 차이가 유난히 큰(상위 5%) 지점에서만 자릅니다 — 자연스러운 화제 전환 경계

⚠️ 처음에 잘 정해야 합니다

자르는 방식은 나중에 변경할 수 없습니다 — 바꾸려면 데이터소스를 새로 만들어 처음부터 전부 다시 색인해야 합니다.

Multimodal Parsing

단순 텍스트(OCR) 추출을 넘어, Claude 모델 기반의 비전 파싱 파이프라인을 적용했습니다. 표, 스캔본 등 복잡한 비구조화 문서 레이아웃을 해석

PART 04

심화 : 커스텀 IDP 구축 전략

AWS Intelligent Document Processing (IDP)

핵심 단순 텍스트 분할을 넘어, 멀티모달 AI 기반의 심층 문서 이해 및 컨텍스트 재생성 파이프라인을 구축하는 방식

Bedrock Knowledge Bases (완전 관리형 RAG)

원시 텍스트 추출 ► 청킹 ► 벡터 보관

빠르고 저렴 — 텍스트 위주의 구조화된 표준 문서 처리에 최적화

AWS IDP (커스텀 전처리 파이프라인)

페이지 이미지 렌더링 ► 비전 AI 추론 ► 분석문+Q&A 저장

AI가 시각적 문맥을 해석하여 재작성한 *시맨틱 메타데이터(요약본 및 Q&A)*를 저장

1	**동적 페이지/길이 단위 처리** *No Traditional Chunking*	토큰 분할 전략을 배제하고, 시각적 문서는 '페이지 단위'로, 텍스트는 컨텍스트 길이에 맞춰 자율적으로 분할
2	**비정형/시각 데이터 한계 극복**	도면, 복잡한 차트, 인포그래픽 등 기존 텍스트 추출(OCR)이 놓치는 시각적 함의를 해석
3	**커스텀 아키텍처 설계** *High Freedom, High TCO*	완성된 서비스형이 아닌 파이프라인 구축형 모델이므로 압도적인 자유도와 커스터마이징을 제공하지만, 운영 및 유지보수 비용이 수반

AWS IDP Use Case

핵심 고도화된 비정형 데이터 파싱 및 엔터티 간 관계성 도출이 비즈니스의 핵심 가치일 경우에 한하여 커스텀 IDP 구축을 권장

1 스캔본 · 저품질 문서 처리

저해상도 스캔, 팩스, 노이즈가 심한 레거시 문서 등 기존 OCR 엔진으로 텍스트 추출이 불가능한 환경에서, 멀티모달 AI의 시각적 추론을 활용

2 도면 · 차트 · 인포그래픽 심층 분석

텍스트가 아닌 이미지 내의 의미를 파악해야만 추론이 가능한 특수 워크로드에 적용

3 다중 문서 간 엔터티 관계 매핑

수많은 문서에 파편화된 인물, 부서, 제품 등의 연관성을 추출하여, Amazon Neptune 기반의 지식 그래프(Knowledge Graph)로 연결하고 다차원 쿼리를 수행해야 할 때

4 대규모 문서 파이프라인 자동화

실시간으로 인입되는 대량의 비정형 문서를 AI가 자율 분류하여 계층적 트리 구조로 적재하고, 전처리 파이프라인 상태를 실시간 오케스트레이션

[TCO(총소유비용) 고려사항] 일반 텍스트 문서의 Q&A가 목적이라면 KB로 충분합니다. KB의 비전 파싱(Claude)만으로도 표 · 스캔본을 상당 부분 커버합니다.

커스텀 IDP 파이프라인 아키텍처 설계

핵심 다수의 인프라 스택을 오케스트레이션하여, 페이지 단위의 멀티모달 심층 분석 파이프라인을 직접 구축

Data Ingestion & Processing Pipeline

1 문서 업로드

비정형 데이터 접수

2 페이지 → 이미지

모든 페이지를 그림으로 변환

3 비전 AI 분석

페이지 단위의 시맨틱 데이터 동적 생성

4 벡터 인덱싱

LanceDB에 적재하여 고속 검색 환경 구성

Core Architecture Components

LanceDB

검색 저장소

임베딩 벡터와 키워드를 결합하고, 리랭킹 (Re-ranking) 알고리즘을 적용하여 KB 대비 검색 정밀도 극대화

Neptune

지식그래프

개별 문서 간의 연관성(인물, 조직 등)을 그래프 데이터베이스로 연결하여 복합 추론(Multi-hop QA) 지원

Workflow Orchestration

분류 · 진행표시

인입된 데이터의 자율 분류 및 실시간 파이프라인 상태 모니터링

[Architecture Trade-off] 페이지 단위의 반복적인 LLM 호출로 인해 연산 오버헤드가 증가하지만, 비정형 데이터의 컨텍스트 손실을 제로화하는 최고 수준의 검색 품질을 보장

Bedrock KB vs 커스텀 IDP 파이프라인

핵심 표준 비즈니스 문서의 대부분은 완전 관리형 KB로 수용하고, 비정형 시각 데이터 처리가 비즈니스 크리티컬한 경우에만 IDP를 제한적으로 도입

	Bedrock KB (현재 프로젝트)	IDP (직접 구축형)
아키텍처 구성	Serverless 기반 턴키(Turn-key) 프로비저닝 (옵션 선택만으로 구성)	14개 이상의 AWS 인프라 스택 직접 오케스트레이션
데이터 파싱 방식	텍스트 추출(OCR) 중심 + 최신 LLM 비전 보조 파싱	모든 페이지 렌더링 기반의 멀티모달 심층 추론
인덱싱 페이로드	원문을 분할한 청크 벡터	AI가 새로 쓴 시맨틱 데이터(분석문 + 예상 Q&A)
Use Case	구조화된 텍스트 및 표준 매뉴얼 기반의 고속·비용 효율적 Q&A	스캔본·도면·차트·인포그래픽
확장성 및 통합	관리형 서비스로 커스터마이징 제한적 (기본 RAG 파이프라인 최적화)	지식그래프 · 자동 분류 · 폴더 정리
총소유비용	매우 우수	높음 (페이지마다 비전 AI 호출)

[도입 전략] 신규 RAG 프로젝트는 도입 리스크가 낮은 Bedrock KB로 시작(PoC)하는 것을 강력히 권장합니다. 최근 고도화된 Advanced Parsing 기능으로 상당수의 특수 문서를 소화할 수 있으며, 이후 한계가 명확한 워크로드에 한해서만 IDP를 점진적으로 결합하는 것이 효과적입니다.

PART 05

Wrap-up 및 실습 안내

성공적인 AI 아키텍처 구현을 위한 4가지 원칙

1

서버리스 기반의 백엔드 모듈화

데이터 파이프라인 및 검색 엔진 운영 등 인프라 영역은 AWS에 맡기고, 개발 조직은 비즈니스 로직과 사용자 경험(UX) 극대화에 집중

2

Kiro 에이전트 주도의 자율형 배포 자동화

인프라 프로비저닝부터 애플리케이션 배포, 검증, 그리고 디버깅까지의 전 과정을 사람이 아닌 **Kiro 에이전트**가 오케스트레이션하여 휴먼 에러를 원천 차단

3

컨텍스트 보존 중심의 데이터 전처리

데이터 분할 정책은 사후 변경이 까다로운 불변 속성이므로, 초기 설계 시 문맥 단절을 방지하는 최적의 전략을 채택하여 검색 품질을 보장

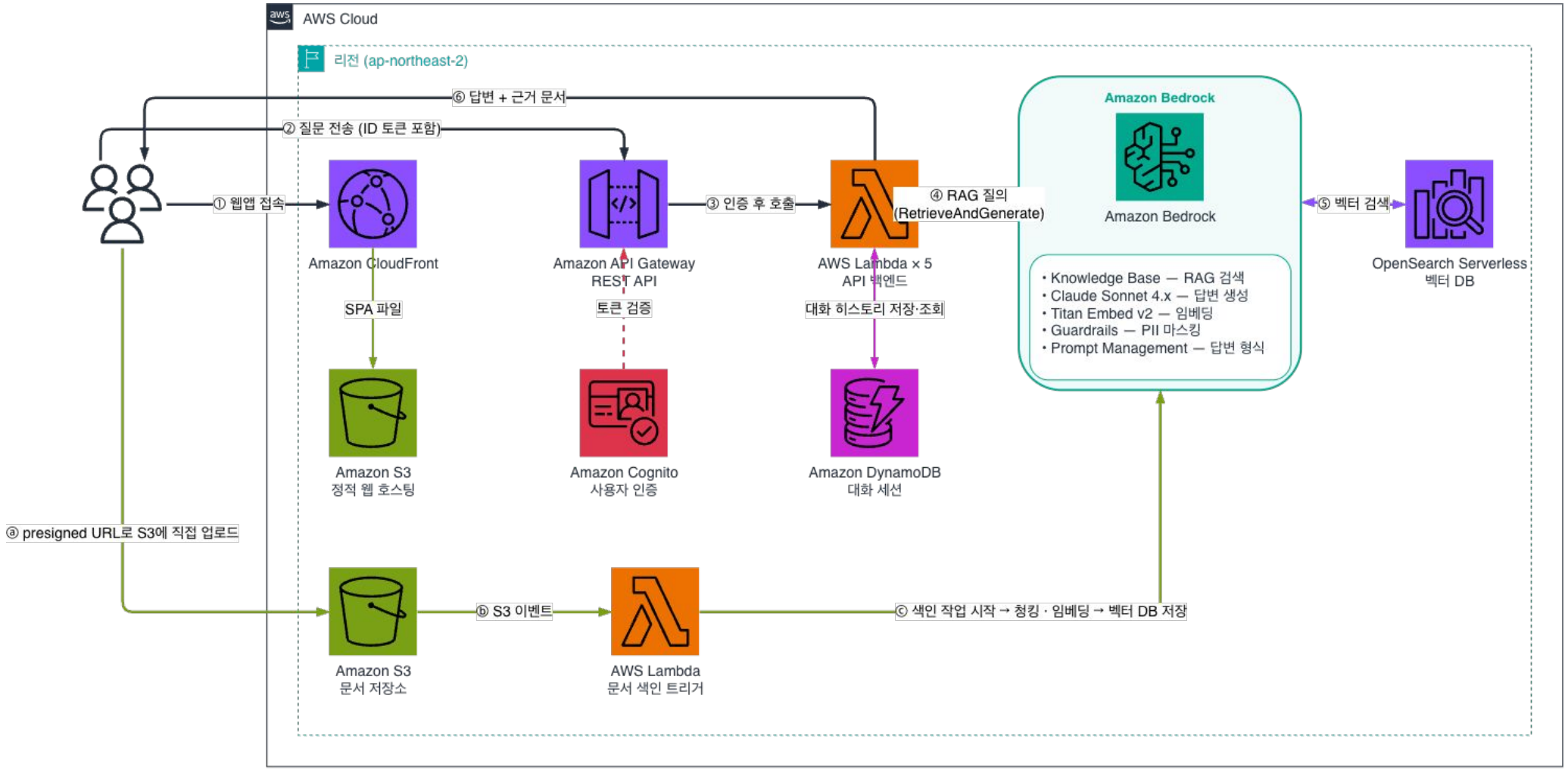
4

엔터프라이즈 RAG의 표준 아키텍처, Bedrock KB

무거운 커스텀 구축(IDP) 없이도 사내 비즈니스 문서를 수용할 수 있는 Bedrock KB를 기본 엔진으로 우선으로 채택하여, 가장 빠르고 안정적인 가치 창출을 실현

서버리스 RAG 챗봇 아키텍처

핵심 Amazon Bedrock Knoewledge Base 기반 문서 질의응답 및 업로드 시 자동 색인



① ~ ⑥ 질의응답 흐름 · ⓐ ~ ⓒ 문서 업로드 · 색인 흐름 · 점선 = 인증 · 제어 경로

AWS CDK(TypeScript)로 전체 인프라 정의 · 4개 CloudFormation 스택으로 배포 · 서버 · 컨테이너 없는 완전 서버리스 구성